

PROJET RENCONTRES

Les Fonctionnalités

- Créer un profil avec ses centres d'intérêt
- Simuler des rencontres entre utilisateurs
- Gagner des points quand on rencontre quelqu'un
- Chercher des gens par intérêts, localisation et nom

Groupe E112

63737 - Fabiola Prenga

63731 - Nicoleta Opre

62098 - Alessian Noje

Filtres de recherche

Filtrer par

Centre d'intérêt

Centre d'intérêt

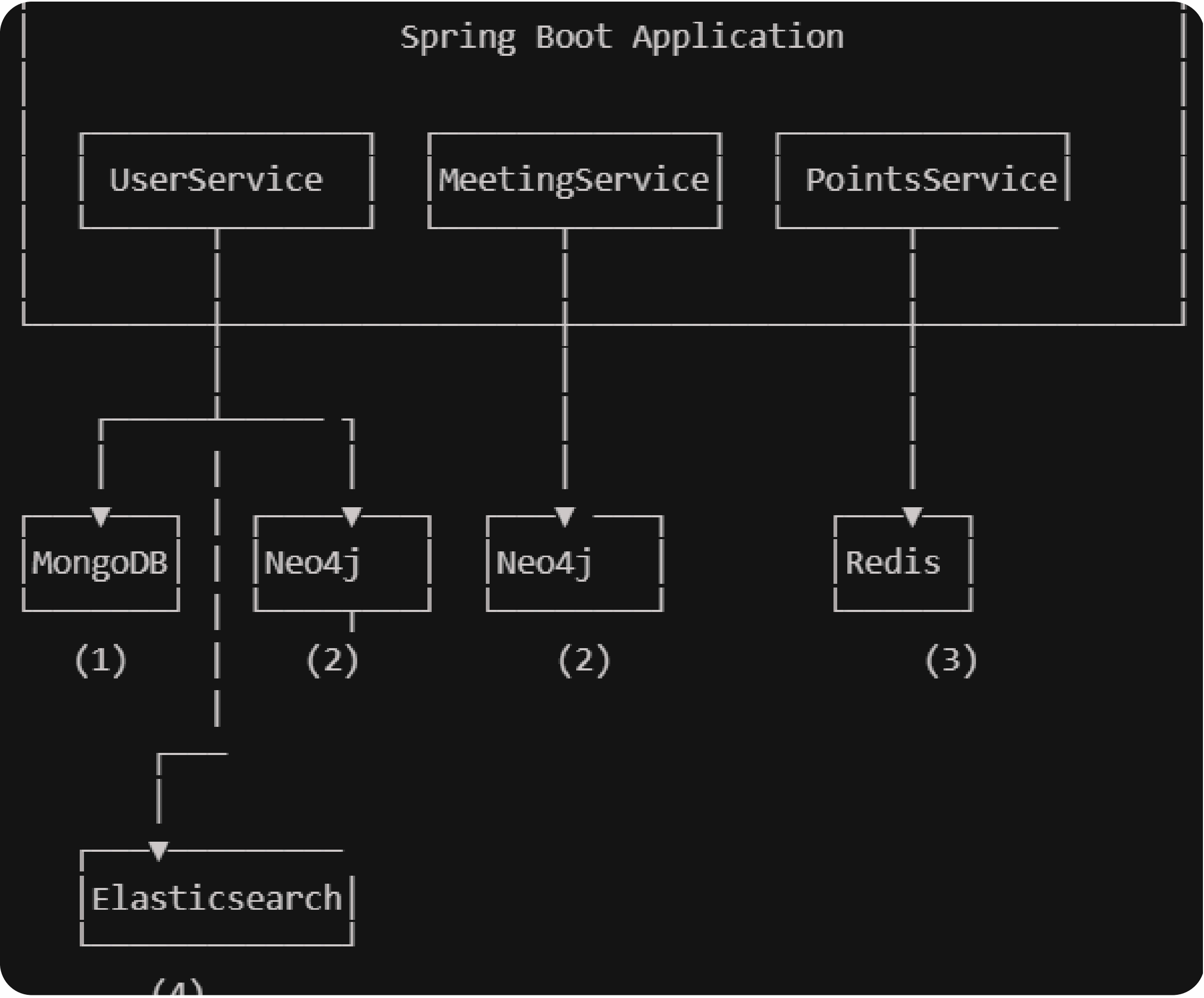
ex: Musique, Sport, Voyage...

Rechercher

 Résultats

Aucune recherche effectuée

ARCHITECTURE GLOBALE



Le système est structuré en 3 services métiers principaux :

Service	Responsabilité	Bases utilisées
UserService	Gestion des utilisateurs et centres d'intérêt	MongoDB, Neo4j, Elasticsearch
MeetingService	Gestion des rencontres entre utilisateurs	Neo4j
PointsService	Gestion des points et classements	Redis

MONGO DB - GESTION DES PROFILS (DOCUMENT)

Rôle : Stockage des informations utilisateurs

Pourquoi MongoDB ?

- **Schéma Flexible** : Les centres d'intérêt varient d'un utilisateur à l'autre. Le format JSON/BSON est parfait pour stocker ces listes sans créer de tables de jointure complexes.
- **Évolutivité** : Si le client décide demain d'ajouter des champs (réseaux sociaux, bio, préférences), on peut le faire sans migrer toute la base.

Structure d'un Document Utilisateur

```
{  
  "_id": "507f1f77bcf86cd799439011",  
  "username": "Alice",  
  "email": "alice@example.com",  
  "bio": "Passionnée de cuisine et de randonnée",  
  "interests": ["cuisine", "randonnée", "photographie"],  
  "localisation": "Bruxelles"  
}
```

Exemples requêtes mongoDB

Exemple 1 : Recherche par centre d'intérêt

```
findByInterestsContaining("cuisine")
```

→ Trouve les utilisateurs qui aiment la cuisine

Exemple 2 : Agrégation - Top centres d'intérêt

```
db.users.aggregate([
  { $unwind: "$interests" },
  { $group: { _id: "$interests", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

→ Liste les centres d'intérêt les plus populaires

- \$unwind : Décompose les listes d'intérêts en documents individuels.
- \$group : Compte les occurrences de chaque intérêt.
- \$sort : Classe les résultats par popularité (décroissant).

NEO4J - GRAPHE DE RELATIONS SOCIALES

Rôle : Modéliser les interactions sociales et l'historique des rencontres.

Pourquoi Neo4j ?

- **Modélisation naturelle des relations** : Représente les utilisateurs comme des nœuds et les rencontres comme des relations. Ce modèle de graphe est plus intuitif et performant que les jointures SQL pour cartographier un réseau social.
- **Requêtes de graphe optimisées** : Grâce au langage Cypher, il est simple et rapide de parcourir les connexions pour des fonctionnalités avancées comme les recommandations ("amis d'amis") ou la recherche de chemins entre personnes.
- **Analyse des réseaux sociaux** : Permet d'analyser la structure du réseau pour trouver les utilisateurs les plus influents ou découvrir des liens indirects entre les membres.

Exemples requettes Neo4j

Exemple : Compter les rencontres d'Alice

```
@Query("MATCH (u:User {id: $userId})-[:MET]-() " +  
        "RETURN count(*) as meetingCount")  
int countMeetingsByUser(@Param("userId") String userId);
```



```
MATCH (u:User {username: "Alice"})-[:MET]-()  
RETURN count(*) as rencontres
```

REDIS – POINTS DE SOCIABILITÉ (CLÉ-VALEUR)

Rôle : Compteur de points en temps réel.

Pourquoi Redis?

- **Performance In-Memory** : Les points de sociabilité peuvent être mis à jour très fréquemment. Redis traite les opérations rapidement
- **Opérations Atomiques** : Utilisation de la commande INCR pour augmenter les points sans risque de "race condition" (conflit si deux rencontres sont enregistrées en même temps).
- **Simplicité** : Pas besoin de structure complexe, une simple clé user:123:points suffit.

Exemples requettes Redis

Exemple 1 : Ajouter 15 points à Bob

```
// Incrémenter le score d'un utilisateur
public void incrementScore(String userId, int points) {
    String key = SCORE_PREFIX + userId;
    redisTemplate.opsForValue().increment(key, points);
}
```

INCRBY score:bob 15
ZADD leaderboard 15 bob

- INCRBY : Met à jour instantanément le compteur individuel de l'utilisateur en mémoire.
- ZADD : Utilise un Sorted Set pour maintenir un classement (leaderboard) mondial sans aucun recalcul lourd.

Exemple 2 : Top 10 du leaderboard

```
// Top N utilisateurs (ZREVRANGE)
public Set<TypedTuple<Object>> getTopUsers(int limit) {
    return redisTemplate.opsForZSet()
        .reverseRangeWithScores(LEADERBOARD_KEY, 0, limit - 1);
}
```

ZREVRANGE leaderboard 0 9 WITHSCORES

- ZREVRANGE extrait le classement des utilisateurs par score décroissant pour afficher le "Top 10" en temps réel avec une performance optimale.

ELASTICSEARCH – MOTEUR DE RECHERCHE (FULL-TEXT)

Rôle : Recherche avancée dans les profils

Pourquoi Elasticsearch ?

- **Recherche Plein Texte** : Permet de trouver "Voyage" même si l'utilisateur a écrit "Voyge"
- **Vitesse** : Indexation inversée qui permet de fouiller dans des millions de profils en quelques millisecondes, là où un LIKE %...% en SQL serait très lent.
- **Découplage** : On ne surcharge pas la base principale (Mongo) avec des requêtes de recherche lourdes.

Exemples requettes Elasticsearch

Exemple 1 : Recherche floue

```
findByUsernameContainingOrBioContaining("cuisne", "cuisne")
```

→ Trouve même avec des fautes de frappe ("cuisne" → "cuisine")

```
// Recherche floue (tolérante aux fautes)
@Query("{\n\"bool\": {\n\"should\": [{\n\"match\": {\n\"interests\": {\n\"query\": \"?0\", \n\"fuzziness\": \"AUTO\"}}}}]}}")
List<UserDocument> findByInterestsFuzzy(String interest);
// Ex: "cuisne" trouve "cuisine"
```

Exemple 2 : Recherche multi-champs

```
// Recherche multi-champs avec boost
@Query("{\n\"multi_match\": {\n\"query\": \"?0\", \n\"fields\": [\n\"username^2\", \n\"bio\", \n\"interests\"]}}")
List<UserDocument> searchMultiField(String query);
```

→ Cherche "cuisine" dans le nom, la bio et les intérêts

RÉPLICATION & SYNCHRONISATION DES DONNÉES

Stratégie : Triple Écriture (Write-Through Pattern)

Lors de la création d'un utilisateur, les données sont répliquées simultanément dans les 3 bases de données :

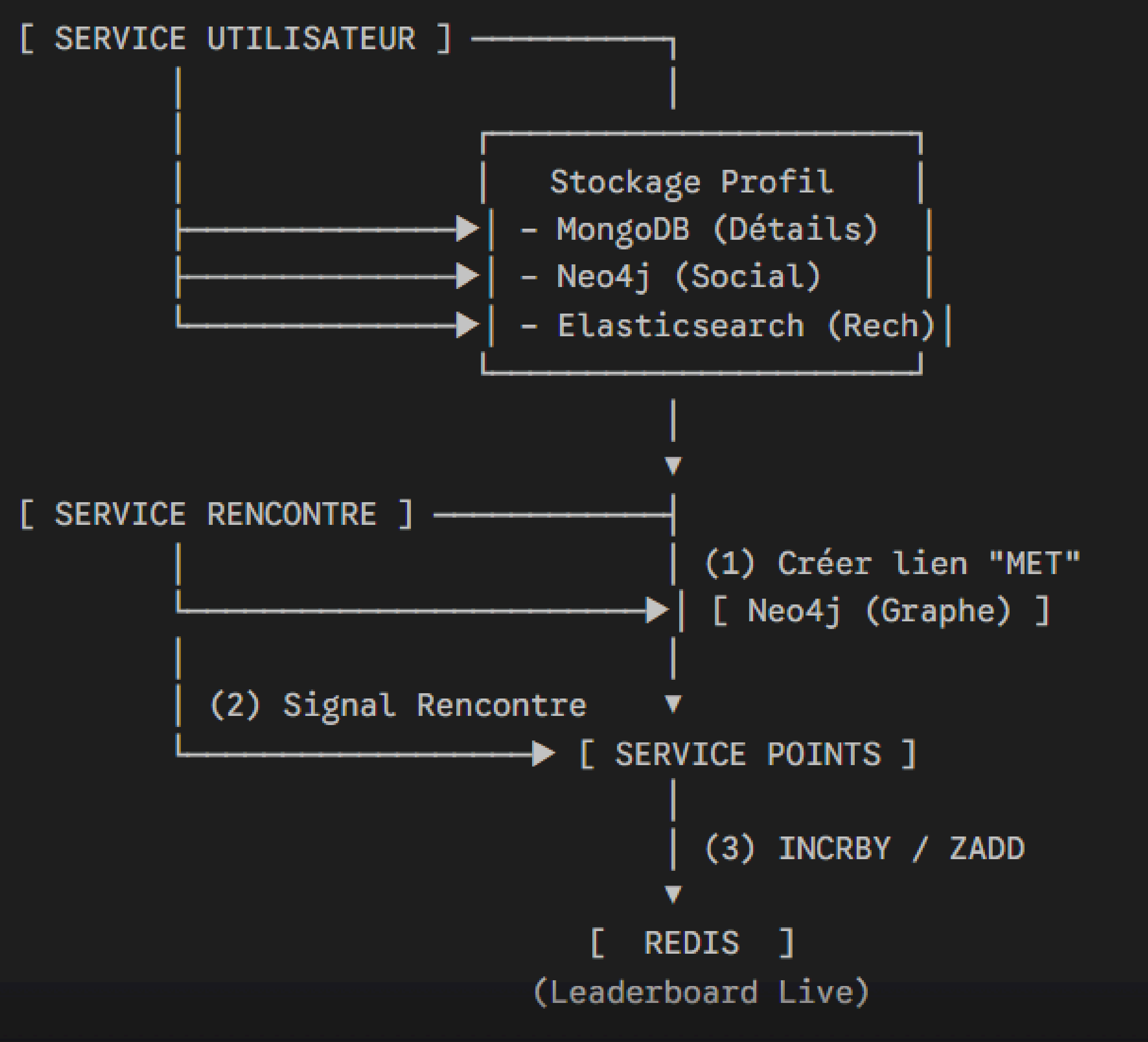
```
public UserDoc registerUser(UserDoc user) {  
    // 1. MongoDB (source de vérité)  
    UserDoc savedUser = userMongoRepository.save(user);  
  
    // 2. Neo4j (graphe)  
    createNeo4jNode(savedUser);  
  
    // 3. Elasticsearch (recherche)  
    indexUserInElasticsearch(savedUser);  
  
    return savedUser;  
}
```

- Chaque création d'utilisateur déclenche 3 écritures séquentielles
- MongoDB → Neo4j → Elasticsearch
- Garantit la cohérence immédiate
- Même ID utilisé dans les 3 bases

Actuellement, en cas d'erreur: si MongoDB réussit mais qu'Elasticsearch échoue, l'utilisateur existe quand même dans MongoDB et Neo4j. La recherche sera temporairement indisponible pour cet utilisateur jusqu'à une réindexation manuelle.

Une amélioration possible serait d'ajouter @Transactional avec un mécanisme de compensation pour faire un rollback complet en cas d'échec partiel. Mais notre approche actuelle garantit déjà la cohérence essentielle : MongoDB reste la source de vérité.

```
@Transactional // Rollback automatique en cas d'échec
public UserDoc registerUser(UserDoc user) {
    try {
        UserDoc savedUser = userMongoRepository.save(user);
        createNeo4jNode(savedUser);
        indexUserInElasticsearch(savedUser);
        return savedUser;
    } catch (Exception e) {
        // Compensation : suppression des données partielles
        rollbackPartialWrites(user.getId());
        throw e;
    }
}
```

Ce schéma illustre la circulation des données entre nos services :

- la création d'un profil alimente simultanément trois bases distinctes, tandis qu'une rencontre enregistrée dans le graphe Neo4j déclenche automatiquement une mise à jour des points dans Redis via un signal inter-service.